Deep Learning
W1: Introduction to Perceptron, MLPs and backpropagation

M. Vazirgiannis

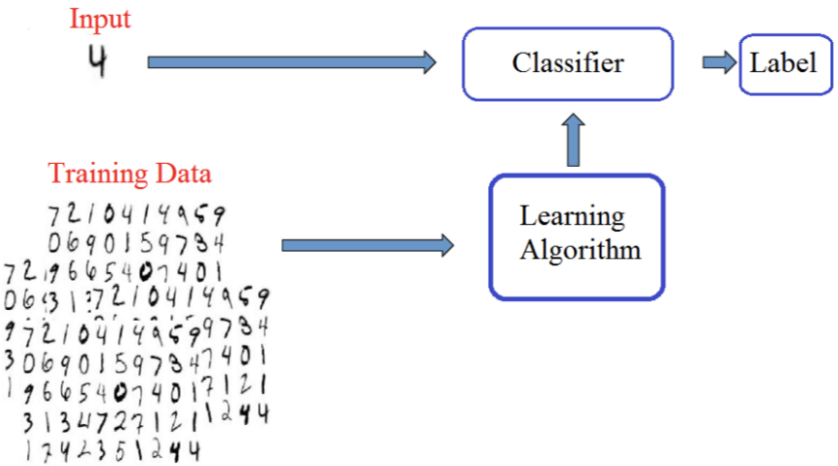
Affiliate Professor, MBZUAI &
Ḋistinguished Professor, Ecole Polytechnique, France
Google Scholar: https://bit.ly/2rwmvQU
Twitter: @mvazirg

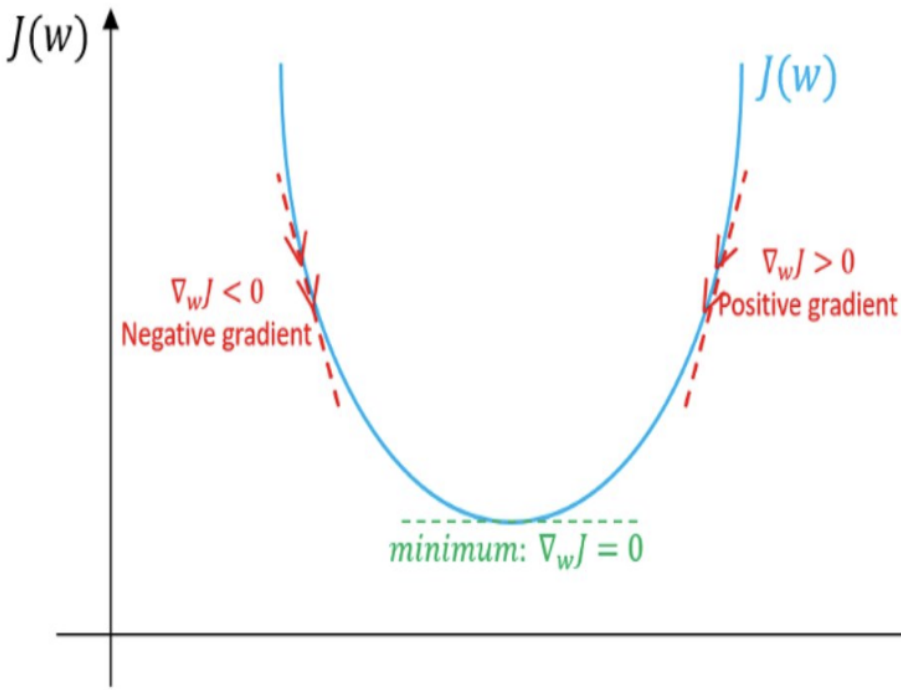
August, 2026

Outline

1 Neural Networks and Deep Learning - Introduction

Machine Learning....

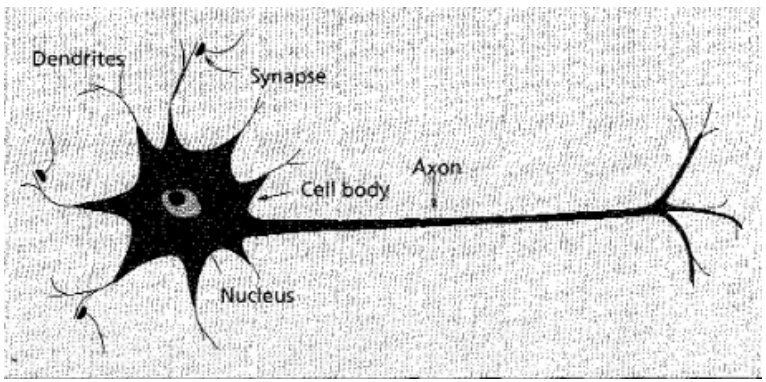




- Data
- Performance
- Model
- Optimization

$$\theta = \theta - \eta \bigtriangledown_\theta J(\theta)$$

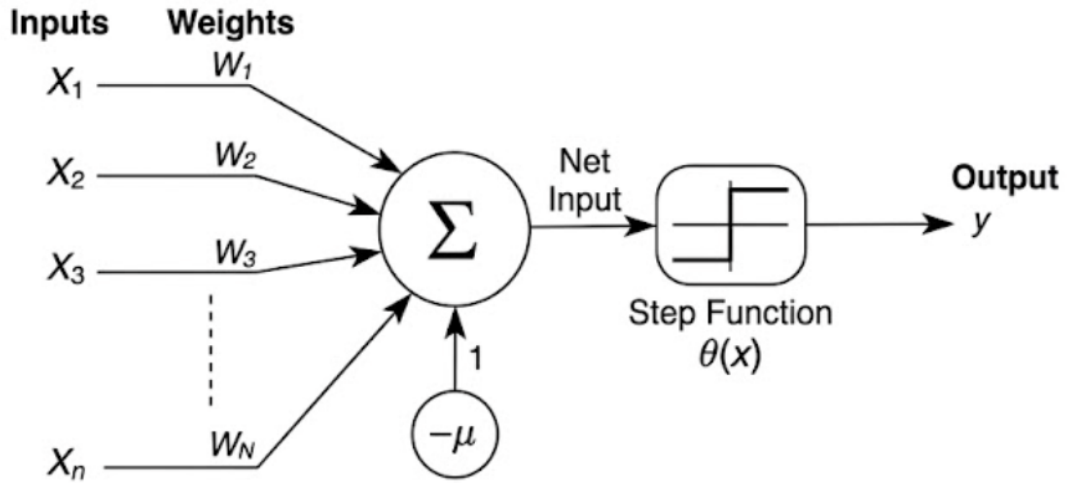
Neural Networks



- Neuron: biological cell processing information
- composed of a cell body, the axon and the dendrites.
- Cell body
  - receives signals (impulses) from other neurons through its dendrites
  - transmits signals generated by its cell body along the axon
- Synapse: functional unit between two neurons (an axon strand and a dendrite)

Artificial neural networks: A tutorial, AK Jain, J Mao, KM Mohiuddin - Computer, 1996

Neural Networks



Rosenblatt Perceptron (1958) Neuron Model $y = \theta(\sum_{j=1}^{n} w_j X_j - \mu)$

- θ: Heaviside step function at 0
- w_j: weight of the j-th input
- μ : threshold (bias $b = -\mu$)
- activation functions: piecewise linear, sigmoid, or Gaussian



Historical Evolution of Neural Architecturess

Year	Model/Architecture	Approx. # Parameters	Application Domains
1943	McCulloch-Pitts Model	Few dozen	Logical reasoning
1958	Perceptron	Hundreds	Pattern recognition
1980s	MLPs & Backpropagation	10^3	Classification & prediction
1990s	CNNs (LeNet-5) - FEATURE EXTRACTION	6*10^4 (LeNet-5)	Image recognition
1997	LSTM - SEQUENTIAL DATA	10^4	Sequential data (speech, text)
2012	AlexNet - CNN	60 * 10^6	Image classification
2014	GANs	10^6	Image generation
2014	Seq2Seq & Attention	10^6	Machine translation
2017	**Transformers - MLP+ Attention**	10*10^6	NLP, Vision
2018	BERT	110*10^6	NLP tasks (e.g., QA)
2020	Vision Transformers	86*10^6	Image classification
2020s	Diffusion Models & CLIP	100*10^6	Multimodal AI
2023	GPT-4V & Multimodal Models	~100*10^9	Multimodal chat, content generation

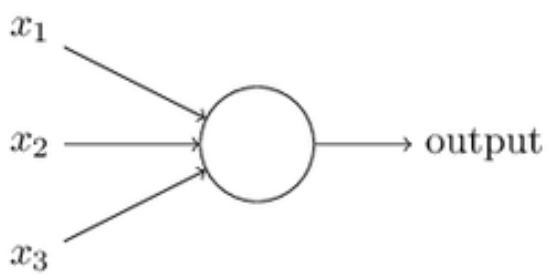
- Artificial NNs - cornerstone for AI evolution
- 2018: Turing Award (G. Hinton, J. Bengio, Y. Lecun).
- massive development since 2010
  - industrial investment
  - abundance of data
  - compute power (GPUs)
- enabled data generation !

Perceptron learning algorithm

- developed in the 1950-60 by Frank Rosenblatt
- takes several binary inputs, x_1, $x_2 \ldots$ produces a single binary output

$$output = \begin{cases} 0 & if \quad \Sigma_j w_j x_j < threshold \\ 1 & if \quad \Sigma_j w_j x_j \geq threshold \end{cases}$$

- Device making decisions by weighing up evidence.
- Varying weights and threshold, we get different models of decision-making
- Equivalently, with bias $b = -threshold$: $output = \Theta\left(\Sigma_j w_j x_j + b\right)$, where $\Theta(z) = \mathbf{1}_{z \geq 0}$ is the Heaviside step



The Rosenblatt Perceptron (1958)

Mathematical Formulation

- Input: $\mathbf{x} \in \mathbb{R}^d$
- Weights: $\mathbf{w} \in \mathbb{R}^d$, Bias: $b \in \mathbb{R}$
- Pre-activation:

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^{d} w_i x_i + b$$

- Step activation (Heaviside):

$$\hat{y} = \Theta(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Learning & Properties

- Update rule for each $(\mathbf{x}_k, y_k)$, where $y_k \in \{0, 1\}$:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left(y_k - \hat{y}_k\right) \mathbf{x}_k$$

$$b \leftarrow b + \eta \left(y_k - \hat{y}_k\right)$$

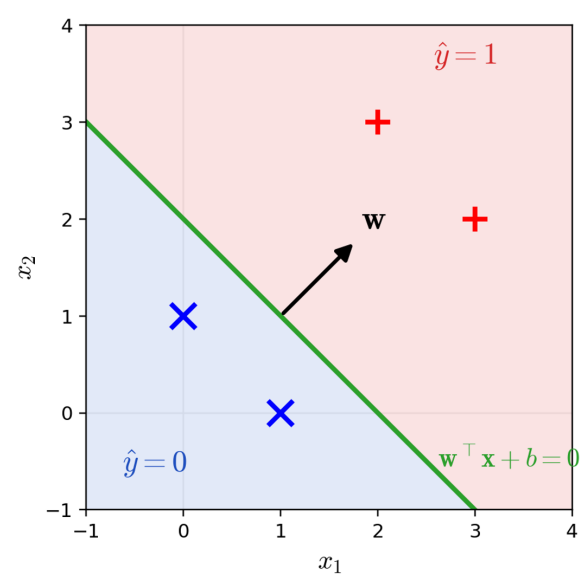
with $\eta > 0$; no update when $\hat{y}_k = y_k$
$k \in 1, N$ (N is the size of training set)

- **Convergence (Novikoff, 1962):** finite if data are linearly separable.

Decision Boundary

- Decision boundary:

$$\mathbf{w}^T\mathbf{x} + b = 0$$

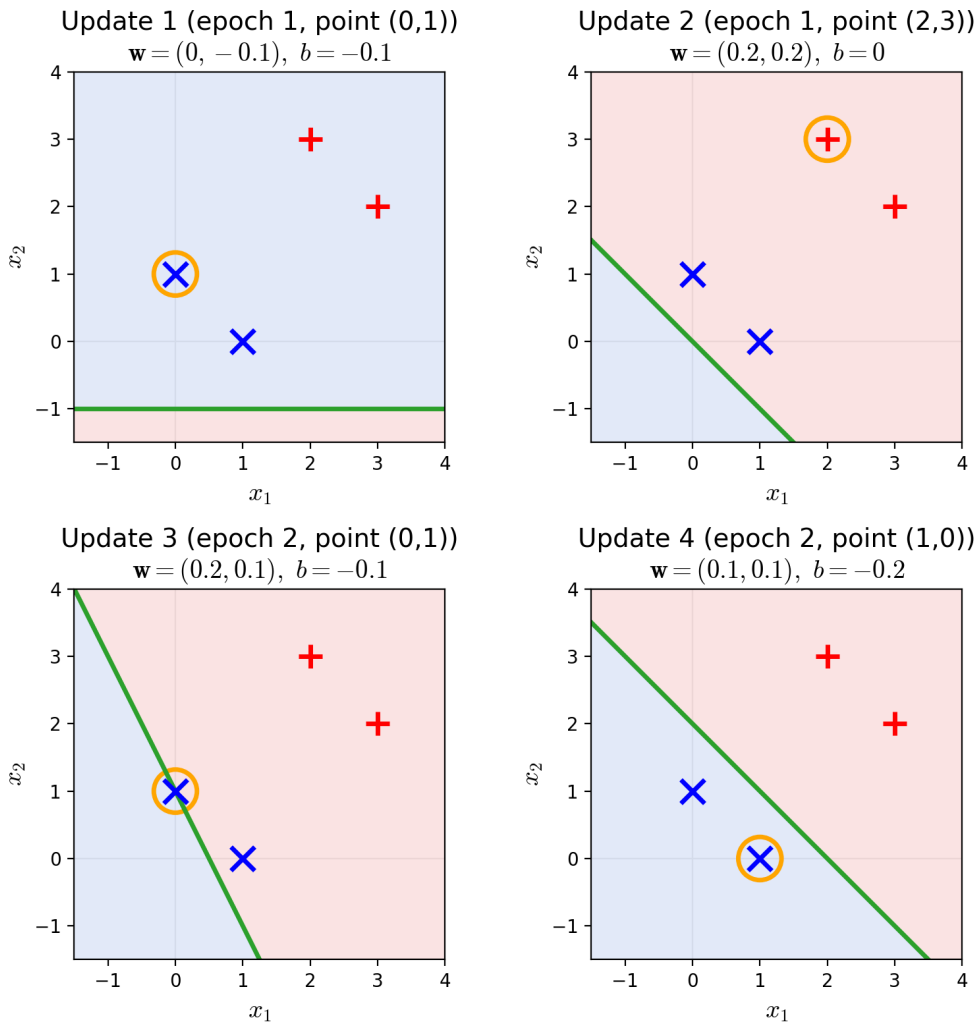


- The perceptron is a linear classifier: the boundary is a line (a hyperplane in M dimensions)
- $\mathbf{x}$ is orthogonal to the boundary and points toward the $\hat{y} = 1$ side
- b shifts the boundary; with $b = 0$ it must pass through the origin

Perceptron learning algorithm

- Initialize weights and bias (small random numbers or zeros)
- Present an input vector $\mathbf{x} = \{x_1, \ldots, x_n\}$ and compute the output $\hat{y} = \Theta\left(\sum_{j=1}^{n} w_j x_j + b\right)$
- Update: $\mathbf{w}_{t+1} = \mathbf{w}_t + \eta(y - \hat{y})\mathbf{x}, \quad b_{t+1} = b_t + \eta(y - \hat{y})$, where η: learning rate, $y \in \{0, 1\}$: correct output
- $\sum_{j=1}^{n} w_j x_j + b = 0$ defines the class separation hyperplane
    - Learning occurs only when the perceptron makes an error

Example: the boundary moves

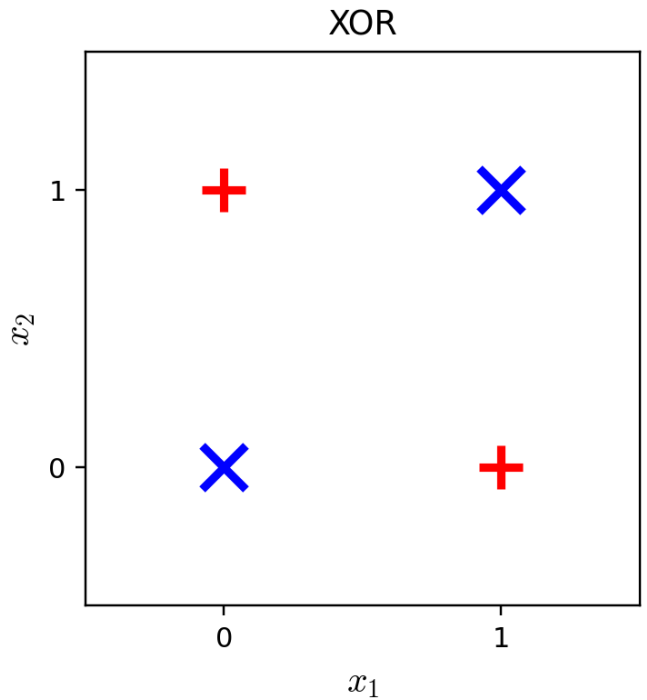


- Orange circle: the misclassified point that caused the update
- Red area: $\hat{y} = 1$; blue area: $\hat{y} = 0$
- Each update moves the line toward the circled point
- **4 updates, 3 epochs**; final boundary $x_1 + x_2 = 2$

Perceptron: convergence and limits

Perceptron convergence theorem (Rosenblatt, 1962): *If training data are drawn from two linearly separable classes, the perceptron learning procedure converges after a finite number of iterations.*

- Small margin: many updates, slow convergence
- Not separable (e.g. XOR): some point is always misclassified, errors never reach 0



Why no line separates XOR: a four-point proof

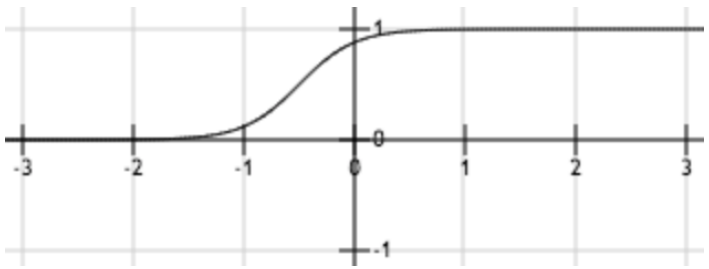
Suppose a perceptron $\hat{y} = \Theta(w_1 x_1 + w_2 x_2 + b)$, with $\Theta(z) = \mathbf{1}_{z \geq 0}$, computes XOR. Each of the four points imposes one constraint:

Input (x_1, x_2)	Target	Constraint on (w_1, w_2, b)
$(0, 0)$	0	(1) $b < 0$
$(1, 0)$	1	(2) $w_1 + b \geq 0$
$(0, 1)$	1	(3) $w_2 + b \geq 0$
$(1, 1)$	0	(4) $w_1 + w_2 + b < 0$

- Add (2) and (3): $w_1 + w_2 + 2b \geq 0$, so $w_1 + w_2 + b \geq -b$
- By (1), $-b > 0$, hence $w_1 + w_2 + b > 0$: **contradicts (4)**
- No choice of (w_1, w_2, b) satisfies all four: the perceptron rule keeps making errors and never stops updating

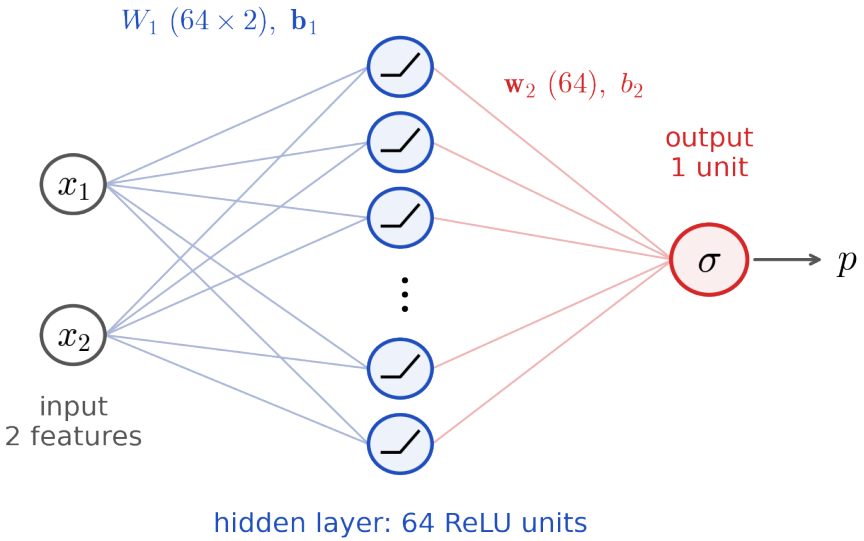
Neural Networks – sigmoid neuron

- In perceptron small changes to the weights may cause huge difference to output ($0 \rightarrow 1$)
- We want to impose: small changes to weights (or bias) small change to the output.
- the sigmoid neuron has weights for each input, $w_1, w_2, \ldots$, and an overall bias b. The output is $\sigma(w \cdot x + \mathrm{b})$, where σ is the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$
- The output of the neuron the is: $\sigma(z) = \frac{1}{1+e^{(-\Sigma_j w_j x_j - b)}}$



Why change the activation? The step function has zero gradient almost everywhere, so gradient descent cannot learn through it; the sigmoid is a smooth version of the step.

Multilayer perceptron (MLP)



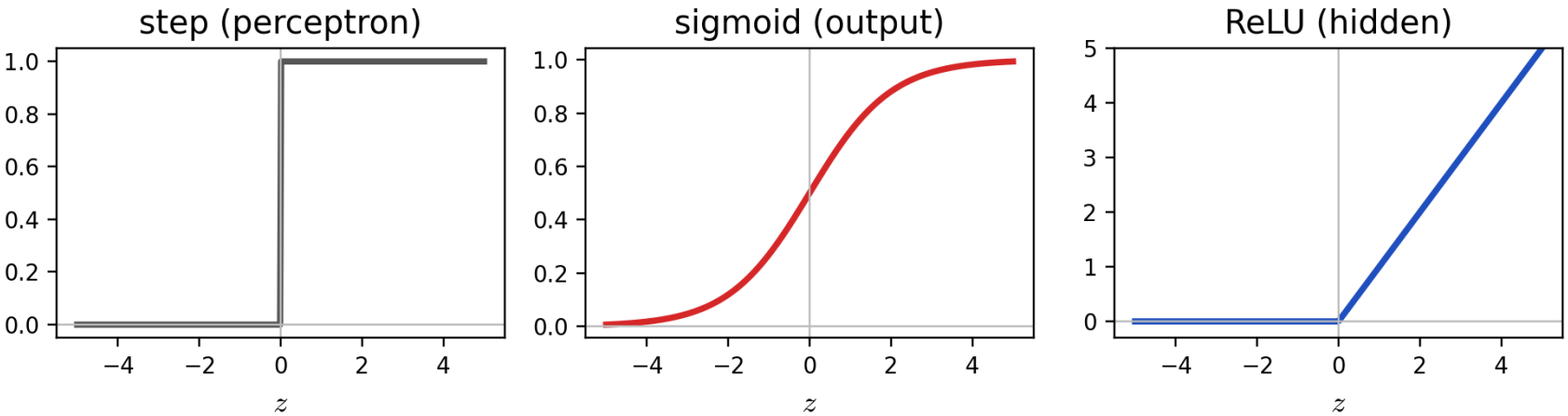
$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1)$$

$$p = \sigma\left(\mathbf{w}_2^\top \mathbf{h} + b_2\right) \in (0, 1)$$

- Hidden layer: 64 units, each a perceptron with a ReLU instead of a step
- Output: 1 unit with a sigmoid; p = probability of class 1
- Prediction: class 1 if $p \geq 0.5$
- **257 parameters**: W_1 (64×2), $\mathbf{b}_1$ (64), $\mathbf{w}_2$ (64), b_2 (1)

Universal Approximation Theorem Cybenko (1989), Hornik (1991): one hidden layer, given enough units, approximates any continuous function to arbitrary precision — depth does this more parameter-efficiently than width alone

Activation functions



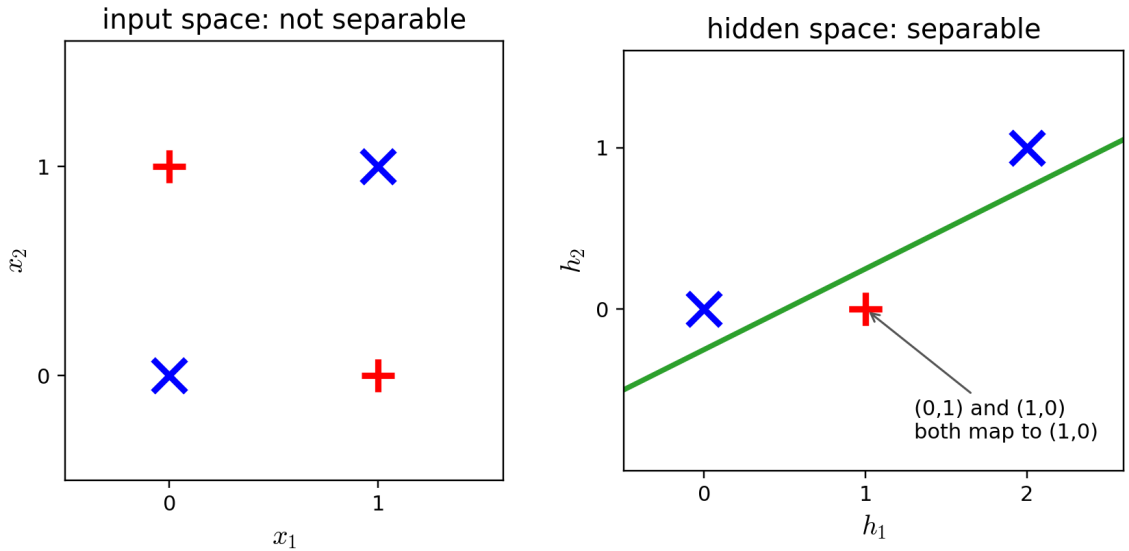
$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \mathrm{ReLU}(z) = \max(0, z)$$

Why a nonlinear activation? Without it, two linear layers collapse into one:

$$\mathbf{w}_2^\top (W_1 \mathbf{x} + \mathbf{b}_1) + b_2 = (\mathbf{w}_2^\top W_1)\, \mathbf{x} + (\mathbf{w}_2^\top \mathbf{b}_1 + b_2) = \mathbf{w}^\top \mathbf{x} + b, \qquad \mathbf{w} = W_1^\top \mathbf{w}_2$$

The network would still draw a line, like the perceptron.

Why a hidden layer helps: XOR



Hidden layer

$$h_1 = \mathrm{ReLU}(x_1 + x_2)$$
$$h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$$

Output

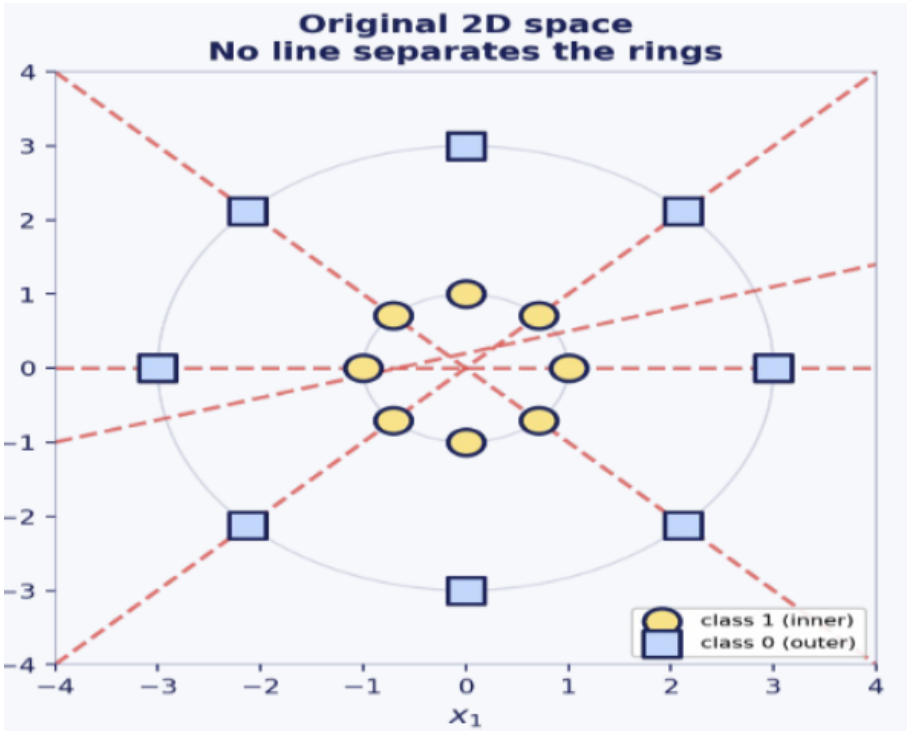
$$\hat{y} = \Theta(h_1 - 2h_2 - 0.5)$$

x	$\mathbf{h} = (h_1, h_2)$	$h_1 - 2h_2 - 0.5$	$\hat{y}$
$(0,0)$	$(0,0)$	-0.5	0
$(0,1)$	$(1,0)$	0.5	1
$(1,0)$	$(1,0)$	0.5	1
$(1,1)$	$(2,1)$	-0.5	0

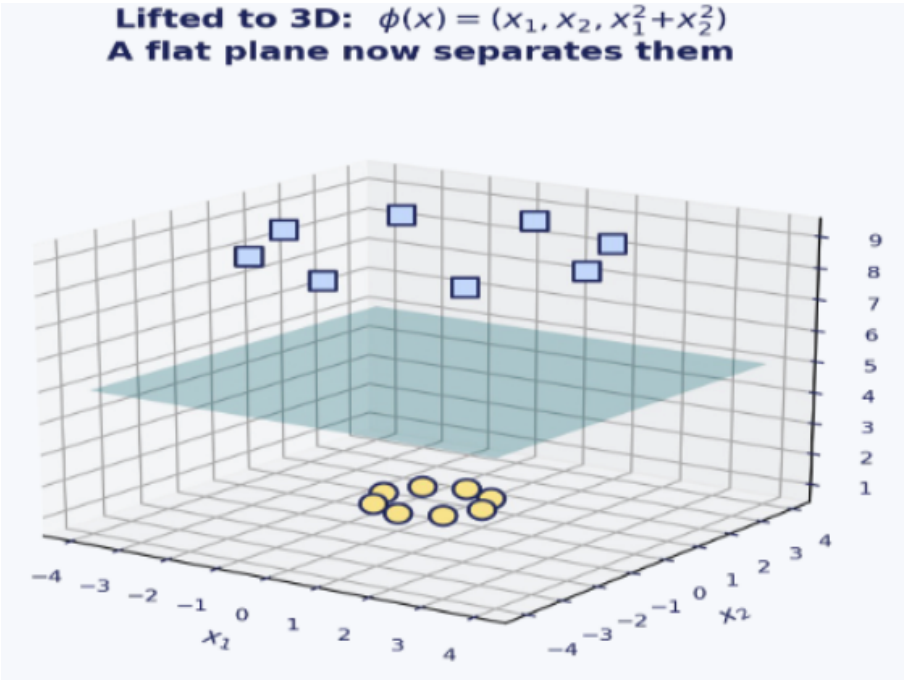
The hidden layer maps the data to a new space where a line separates the classes.

Lift into a higher dimension

Original 2D space
No line separates the rings



Lifted to 3D: $\phi(\mathbf{x}) = (x_1,\ x_2,\ x_1^2 + x_2^2)$
A flat plane now separates them

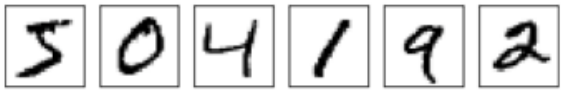


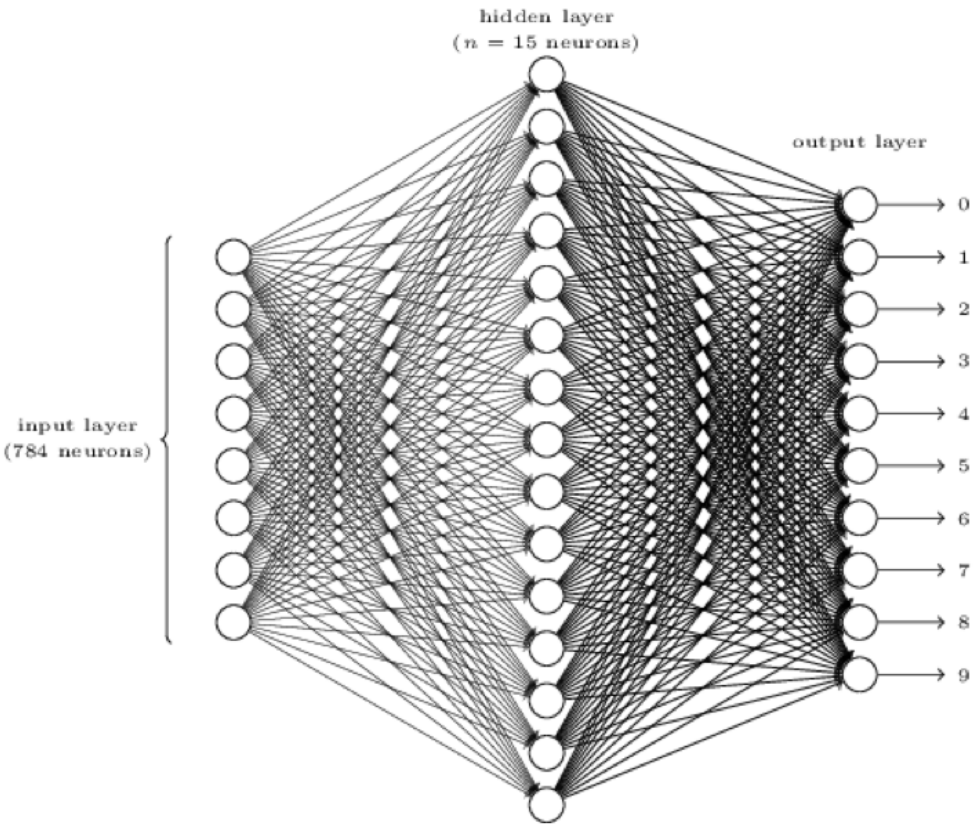
$\phi(\mathbf{x}) = (x_1, x_2, x_1^2 + x_2^2)$ **is a hand-picked feature map, (kernel)** — a hidden layer learns a nonlinear higher dimension automatically from data.

Cover's theorem (1965): *patterns are more likely to become linearly separable when embedded into higher dimensions.*

Learning with gradient descent

- Assume hand written digits (MNIST database – 60.000 images):
- Each of them is a 28x28=784 pixels





- Assume a NN with 784 inputs, a hidden layer of 15 neurons and 10 outputs: i.e.: if x is an image (=7) $y(x)$: $(0, 0, 0, 0, 0, 0, 0, 1, 0, 0)$ is the desired output.

http://neuralnetworksanddeeplearning.com/chap1.html

Learning with gradient descent

- Cost/Loss function $L(w, b) = \frac{1}{2n}\Sigma_x |y - f(x)|^2$
  - w: weights, b: biases, y: vector of correct outputs, n: total number of training samples
- Searching for w, b to minimize $L(w, b)$

$$w_k^{'} = w_k - \eta \frac{\partial L}{\partial w_k}$$

$$b_l^{'} = b_l - \eta \frac{\partial L}{\partial b_l}$$

Problems with GD: $L = \frac{1}{n}\Sigma_x L_x$ where $C_x = \frac{\|y - f(x)\|^2}{2}$

To compute the gradient ∇L we must compute gradients ∇L_x for each training point and then average them: $\nabla L = \frac{1}{n}\Sigma_x \nabla L_x$

$\Rightarrow$ very slow learning

Learning with stochastic gradient descent (SGD)

- estimate ∇L by computing ∇L_x for of randomly chosen samples of training points
- averaging over sample: estimate of the true gradient ∇L - faster
- Chose m random subsets (with replacement) of the training set: $X_1, X_2, \ldots, X_m$ - mini-batches.
- Assuming sample size m large enough we expect that the average value of the ∇L_{X_j} will be roughly equal to the average over all ∇L_x:

$$\frac{\Sigma_{j=1}^{m} \nabla L_{X_j}}{m} \approx \frac{\Sigma_x \nabla L_x}{n} = \nabla L$$

Learning with stochastic gradient descent (SGD)

- w_k and b_l: weights and biases of the perceptron connections in the neural network. SGD trains on randomly choses a mini-batch of training points.

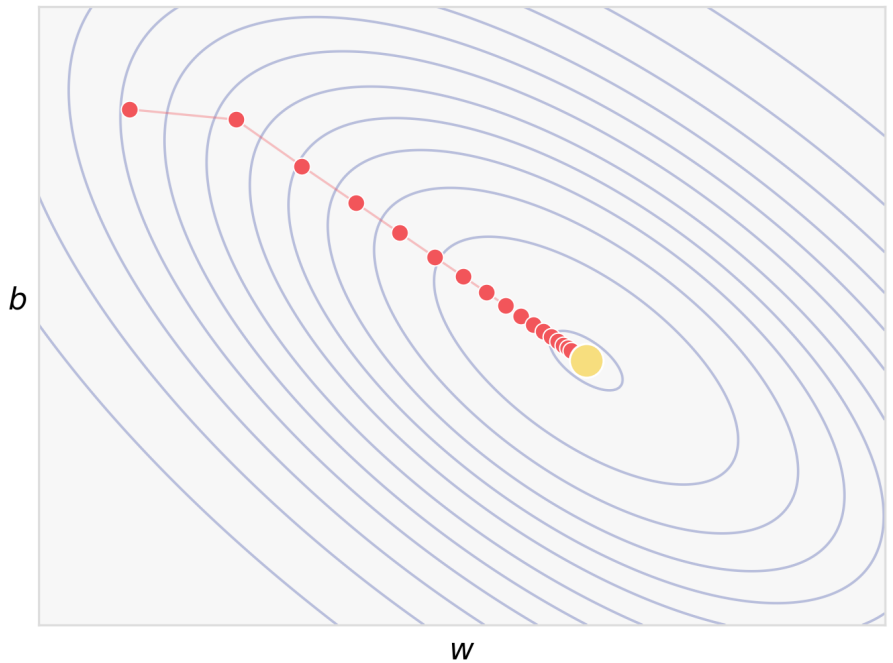
$$w_k \rightarrow w_k^{'} = w_k - \frac{\eta}{m} \Sigma_j \frac{\partial L_{X_j}}{\partial w_k}$$

$$b_l \rightarrow b_l^{'} = b_l - \frac{\eta}{m} \Sigma_j \frac{\partial L_{X_j}}{\partial b_l}$$

- X_j: training examples in the current mini-batch.
- pick another randomly chosen mini-batch and train with those.
- until all training points used (complete an *epoch* of training).

Gradient descent vs Stochastic GD

- Before taking a step, GD looks at the *entire* dataset to compute the exact slope (gradient) of the loss.
- GD uses all N examples, each update points straight toward steepest descent.
- **expensive**: for 10^6 points, to calculate the gradient we need a full pass over all of them

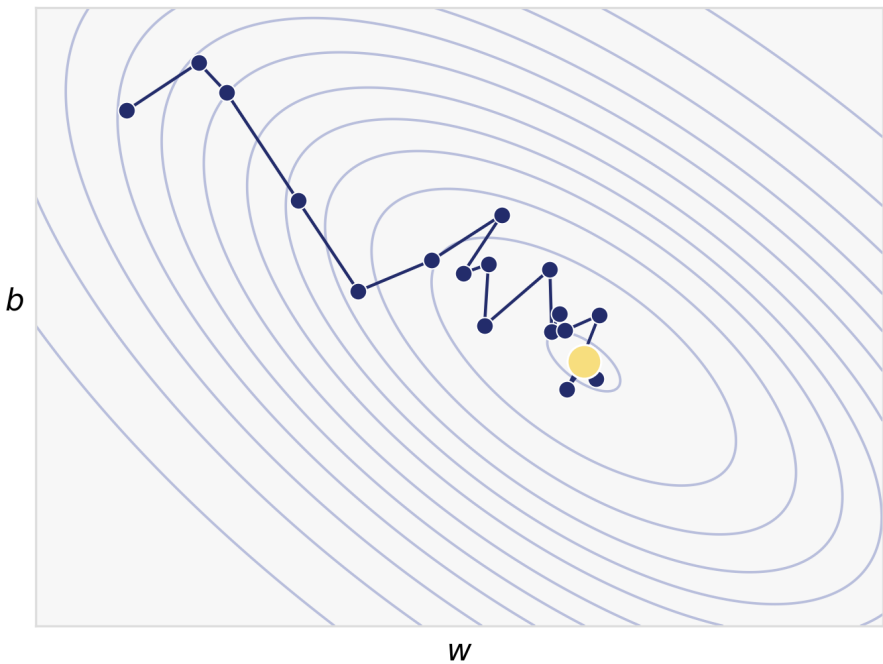


Update rule: $w \leftarrow w - \eta \cdot \nabla L(w;\ \text{all } N \text{ points})$

η = learning rate, ∇L = gradient of the loss averaged over the entire dataset

Stochastic Gradient Descent

- SGD estimates slope from *one* random example or a small mini-batch $\mathcal{B}, |\mathcal{B}| = k << N$ instead of entire dataset.
- **Noisy but frequent**: each step a rough guess of true direction, thus path zig-zags and do many more steps.
- **Cheap and scalable**: when N is huge: we don't need a full pass before updating the weights.

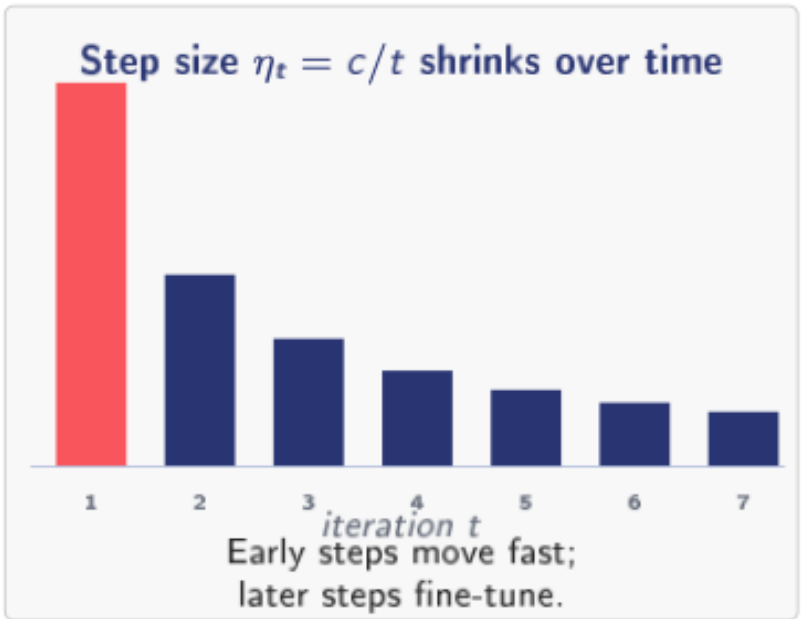


Update rule: $w \leftarrow w - \eta \cdot \nabla L(w;$ one point $i)$

, i picked at random and reshuffled each epoch

Why does SGD converge at all?

- **Each step is unbiased**: Pick a random index i . On average, its gradient IS the true gradient: $\mathbb{E}[\nabla L_i(w)] = \nabla L(w)$
- **Progress minus a noise penalty**: For an L-smooth loss, one SGD step obeys:
  $\mathbb{E}[L(w_{t+1})] \leq$
  $\mathbb{E}[L(w_t)] - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla L(w_t)\|^2 + \frac{\eta^2 L\sigma^2}{2}$
  penalty from gradient noise σ^2
- **Shrink the step size to tame the noise**: A decaying learning rate (e.g. $\eta_t = c/t$) must satisfy Robbins–Monro conditions:
  $\sum_t \eta_t = \infty$ — steps never stop, so any point is reachable
  $\sum_t \eta_t^2 < \infty$ — steps shrink fast enough for the noise to vanish



Letting SGD settle exactly onto the minimum instead of jittering around it.

How fast does each converge?

	Gradient Descent	Stochastic GD
Convergence rate	Linear: $(1 - \mu/L)^t$	Sublinear: $\mathcal{O}(1/t)$
Iterations for ε-accuracy	$\mathcal{O}(\log 1/\varepsilon)$	$\mathcal{O}(1/\varepsilon)$
Cost per iteration	$\mathcal{O}(N)$ — full dataset	$\mathcal{O}(1)$ — one example
Total compute to reach ε	$\mathcal{O}(N \log 1/\varepsilon)$	$\mathcal{O}(1/\varepsilon)$

Parameters (L-smooth, μ-strongly convex):

- L **(Smoothness / Max Curvature):**
  $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$
  $\rightarrow$ Dictates step size ($\eta \leq 1/L$).
- μ **(Strong Convexity / Min Curvature):**
  $f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \frac{\mu}{2}\|y - x\|^2$
  $\rightarrow$ Ensures quadratic growth from minimum.

Takeaways:

- **GD:** Far fewer iterations, but each step scales as $\mathcal{O}(N)$.
- **SGD:** More iterations, but each step is $\mathcal{O}(1)$ (independent of N).
- **Result: SGD wins at scale!**

GD vs SGD - Comparison

	Gradient Descent	Stochastic GD
Data used per update	All N examples	1 example (or a small mini-batch)
Path to the minimum	Smooth, direct	Noisy, zig-zagging
Compute per step	High (full pass)	Low (one example)
Updates per epoch	1	N
Best suited for	Small / medium datasets	Large-scale, streaming data

In practice: mini-batch SGD (32 - 256 examples per step) is smoother than pure SGD, cheaper than full-batch GD.

GD vs SGD – An example[1]

x	1	2	3	4
y **(true)**	3	5	7	8
$\hat{y} = w \cdot x$	1	2	3	4
Gradient $2x(\hat{y} - y)$	-4	-12	-24	-32

Gradient Descent (uses all 4 points):

$$\text{avg gradient} = \frac{-4 - 12 - 24 - 32}{4} = -18$$

$$w_{\text{new}} = 1 - 0.01 \times (-18) = 1.18$$

Stochastic GD (1 random point, e.g., $i = 3$):

$$\text{gradient at point } 3 = -24 \text{ (no averaging)}$$

$$w_{\text{new}} = 1 - 0.01 \times (-24) = 1.24$$

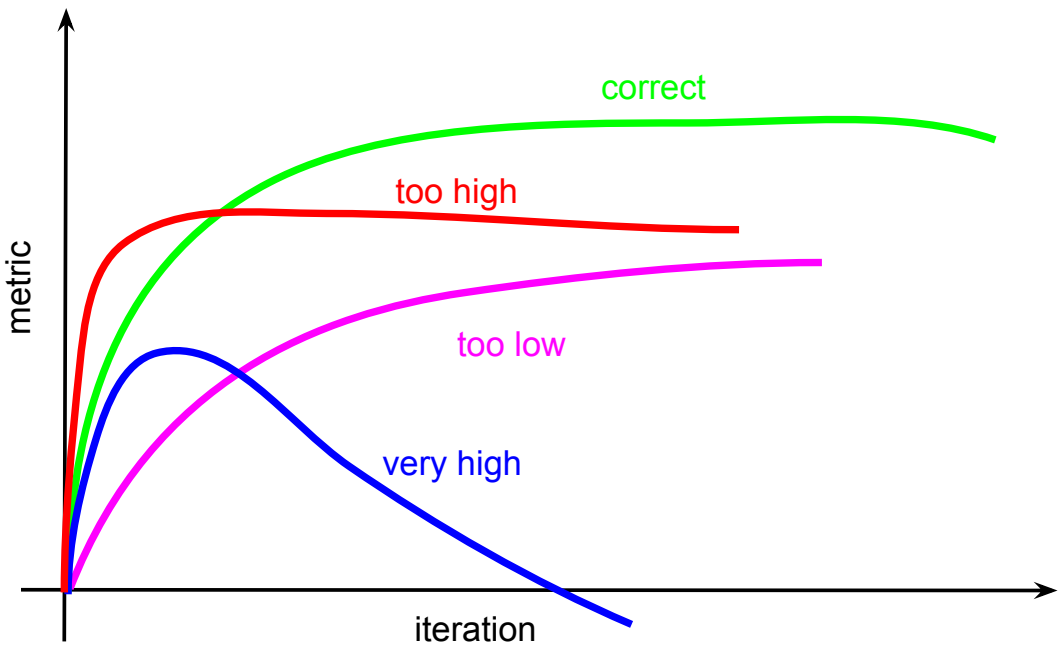
Takeaway

- **GD's step** (-18): true average direction — accurate, but needed all 4 points.
- **SGD's step** (-24): noisy estimate from one point — cheaper, available instantly.
- Over many random points, SGD's steps average to roughly same direction as GD.
- Trade-off: exactness and smoothness vs. speed and scale.

[1] Fitting $y \approx w \cdot x$ with a tiny dataset ($\eta = 0.01$, current $w = 1$)

Learning Rate

- In vanilla SGD learning rate is fixed (this version is still used today)
- but we need to tune the learning rate (which is an *hyperparameter* of the model)
- Typically try: $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$.



How to evaluate the learning rate.

How do MLPs learn? - Backpropagation

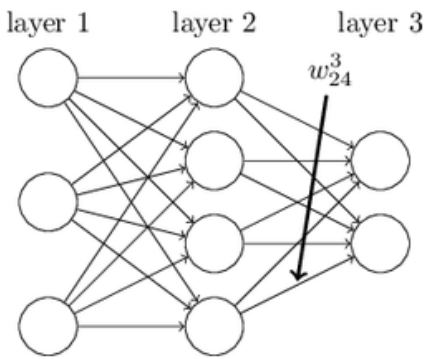
- Learning parameters with gradient descent

$$w_k^{'} = w_k - \eta \frac{\partial C}{\partial w_k}$$

$$b_l^{'} = b_l - \eta \frac{\partial C}{\partial b_l}$$

- How to compute the gradients: backpropagation

  w_{jk}^{l} : weight among the kth neuron of the $l-1$ layer to the jth neuron of the l layer.



Letters to Nature, Nature 323, 533-536 (9 October 1986) | doi:10.1038/323533a0; Learning representations by back-propagating errors, David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams..

The backpropagation algorithm

- **Input x**: Set the corresponding activation α^l for the input layer.
- **Feedforward:** For each $l = 2, 3, \ldots, L$ compute $z^l = w^l \alpha^{l-1} + b^l$ and $\alpha^l = \sigma(z^l)$.
- **Output error** $delta^L$: Compute the vector $\delta^L = \nabla_a C \odot \sigma'(z^L)$
- **Backpropagate error**: For each $l = L-1, L-2, \ldots, 2$ compute

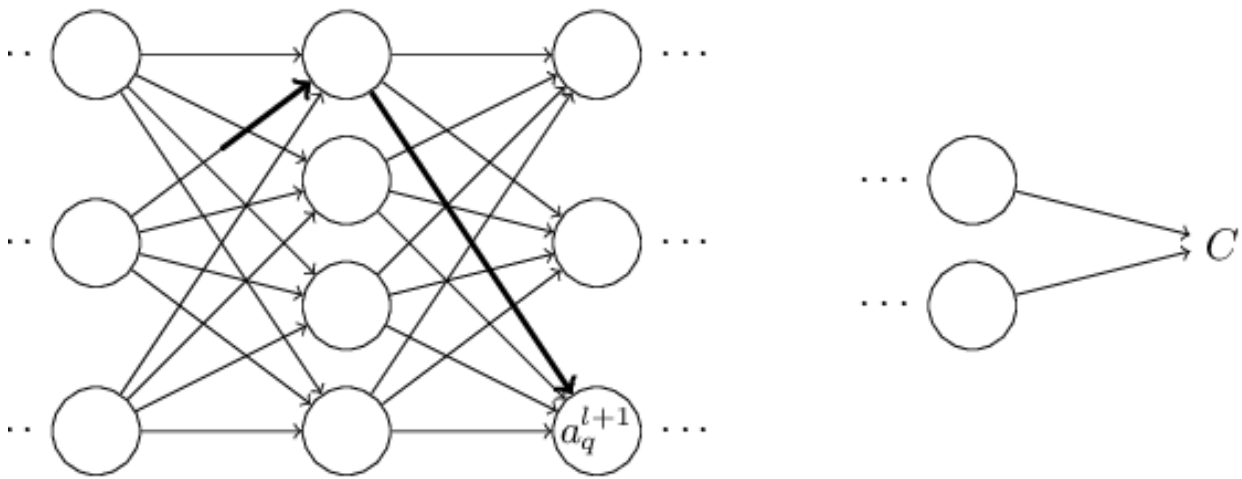
$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^L).$$

- **Output**: gradient of the cost function:

$$\partial C / \partial w^l_{jk} = a^{l-1}_k \delta^l_j$$
$$\partial C / \partial b^l_j = \delta^l_j$$

Back propagation - the big picture

- Δw_{lj}^k : small change in weight w_{lj}^k
- causes change of the output activation from the corresponding neuron: $\Delta \alpha_j^l \approx \frac{\partial \alpha_j^l}{\partial w_{jk}^l} \Delta w_{jk}^l$
- causing changes in all activations in the next layer
- Thus: $\Delta \alpha_q^{l+1} \approx \frac{\partial \alpha_q^{l+1}}{\partial \alpha_j^l} \Delta \alpha_j^l$

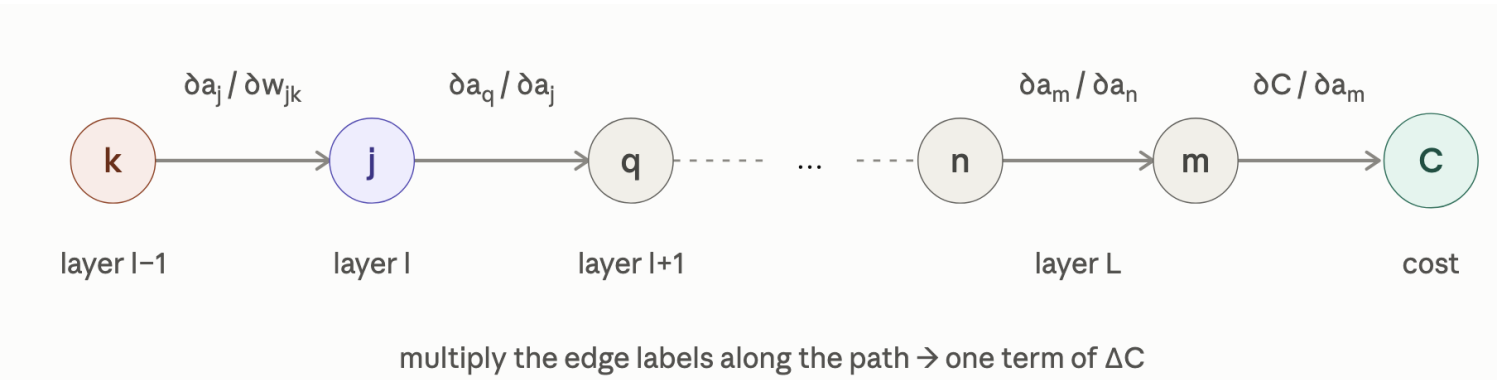


Back propagation - the big picture

Cascading changes to next layers cause change in the output:

- Single path:

$$\Delta C \approx \frac{\partial C}{\partial a_m^L} \frac{\partial a_m^L}{\partial a_n^{L-1}} \frac{\partial a_n^{L-1}}{\partial a_p^{L-2}} \cdots \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l} \Delta w_{jk}^l$$

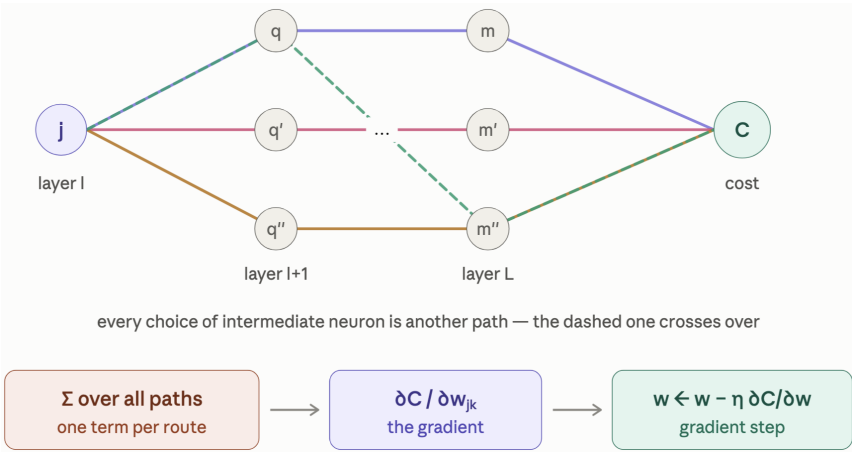


Back propagation – the big picture

Cascading changes to next layers cause change in the output:

- **Sum over all paths:**

$$\Delta C \approx \sum_{mnp\ldots q} \frac{\partial C}{\partial a_m^L} \frac{\partial a_m^L}{\partial a_n^{L-1}} \frac{\partial a_n^{L-1}}{\partial a_p^{L-2}} \cdots \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l} \Delta w_{jk}^l$$



- **Effect of weight change on total cost:**

$$\frac{\partial C}{\partial w_{jk}^l} \approx \sum_{mnp\ldots q} \frac{\partial C}{\partial a_m^L} \frac{\partial a_m^L}{\partial a_n^{L-1}} \frac{\partial a_n^{L-1}}{\partial a_p^{L-2}} \cdots \frac{\partial a_q^{l+1}}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{jk}^l}$$

- **Weights update:**

$$w_k' = w_k - \eta \frac{\partial C}{\partial w_k}$$

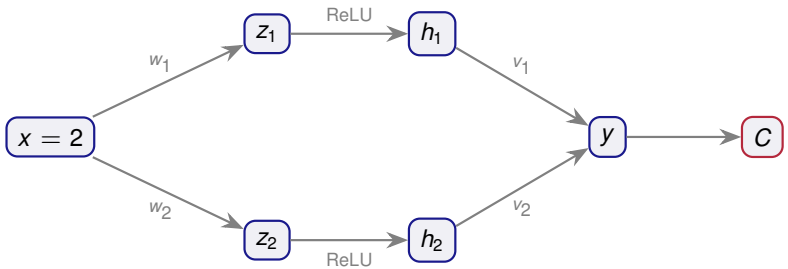
$$b_l' = b_l - \eta \frac{\partial C}{\partial b_l}$$

Computation Graph Example

$$z_1 = w_1 x + b_1, \quad h_1 = \mathrm{ReLU}(z_1)$$
$$z_2 = w_2 x + b_2, \quad h_2 = \mathrm{ReLU}(z_2)$$
$$y = v_1 h_1 + v_2 h_2, \quad C = \tfrac{1}{2}(y - t)^2$$

x	t	w_1	b_1	w_2	b_2	v_1	v_2
2	3	0.5	-0.5	-1	3	2	3

Node	Expression	Value
z_1	$0.5 \cdot 2 - 0.5$	**0.5**
h_1	$\max(0, 0.5)$	**0.5**
z_2	$-1 \cdot 2 + 3$	**1.0**
h_2	$\max(0, 1.0)$	**1.0**
y	$2(0.5) + 3(1.0)$	**4.0**
C	$\tfrac{1}{2}(4 - 3)^2$	**0.5**



- Which values must be **stored** for the backward pass?

(Answer: h_1, h_2 and the *signs* of z_1, z_2 : one bit per ReLU unit. Note the memory cost of training.)

What Does "Differentiable" Mean?

A deep learning framework is **differentiable** because it automatically evaluates gradients without manual analytic derivations.

The Autodiff Mechanism:

- **Not Symbolic:** Does not manipulate algebraic expressions (e.g., Mathematica).
- **Not Numerical:** Does not approximate via finite differences ($\frac{f(x+\epsilon)-f(x)}{\epsilon}$).
- **Automatic Differentiation:** Evaluates **exact derivatives** by decomposing computations into elementary operations and applying the chain rule step-by-step.

The Node Contract: Every registered primitive operation implements a dual interface:

1. **Forward Pass:** Computes output values and caches inputs (memoization).
2. **Backward Pass:** Computes vector-Jacobian products (local gradients).

The Computation Graph & Primitive Primitives

During the forward pass, the runtime builds a directed acyclic graph (DAG). The backward pass traverses this graph in reverse order using the product rule.

Operation	**Forward Function**	**Backward Derivative**
Addition	$c = a + b$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c}$
Multiplication	$c = a \cdot b$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c} \cdot b, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial c} \cdot a$
Sigmoid	$y = \sigma(x)$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \sigma(x)(1 - \sigma(x))$
ReLU	$y = \max(0, x)$	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \mathbf{1}_{\{x>0\}}$
Matrix Multiply	$Y = WX$	$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y} X^\top, \quad \frac{\partial L}{\partial X} = W^\top \frac{\partial L}{\partial Y}$

Practical Execution & Why It Matters

```
import torch

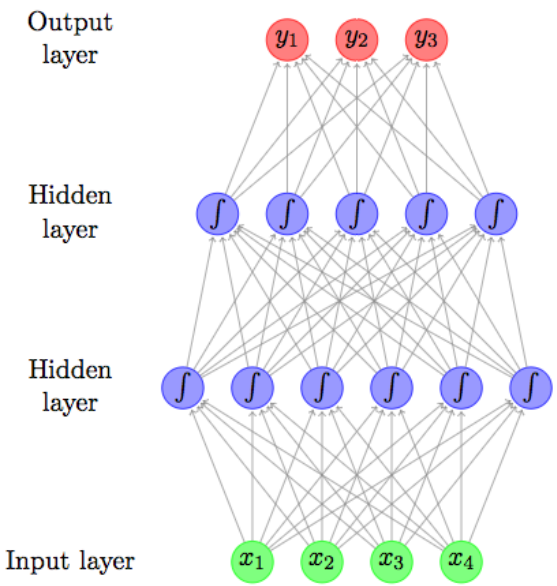
x = torch.tensor([0.5, 0.3], requires_grad=True)
h = x * 2 + 1      # Forward graph dynamically constructed
loss = h.sum()

loss.backward()    # Exact gradients via reverse-mode traversal
print(x.grad)      # Outputs: tensor([2., 2.])
```

Why This Matters in Practice:

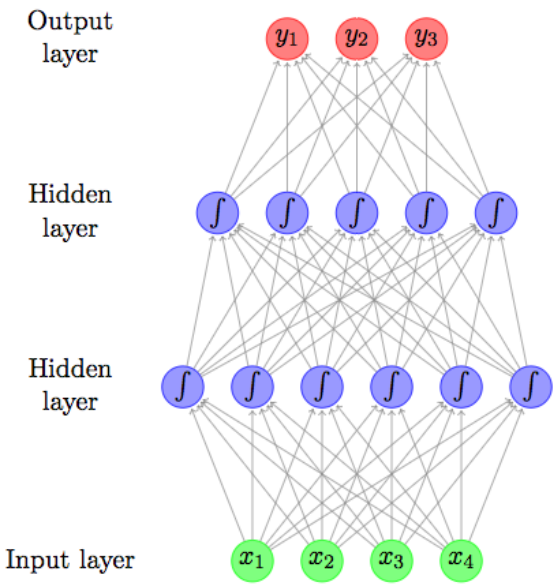
- **Decoupled Architecture Design:** Modify loss functions, add layers, or integrate non-Euclidean graph layers without re-deriving backpropagation equations.
- **Unified Optimization Pipeline:** Calling `loss.backward()` handles arbitrary compositional complexity automatically.

Multilayer Feed-forward NNs



- case: 3 class classification
- top layer: output of the network (one-hot encoded).
- other layers are considered “hidden”.
- hidden layer neurons represent non-linear functions.
- applied to neuron’s value before passing it to the output.
- fully-connected: each neuron connected to all next layer neurons.

Multilayer Feed-forward NNs



- **x**: input
- **h**: output of first hidden layer
- **y**: output

$$NN_{MLP2}(\mathbf{x}) = (g^2(g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)\mathbf{W}^2 + \mathbf{b}^2))\mathbf{W}^3$$

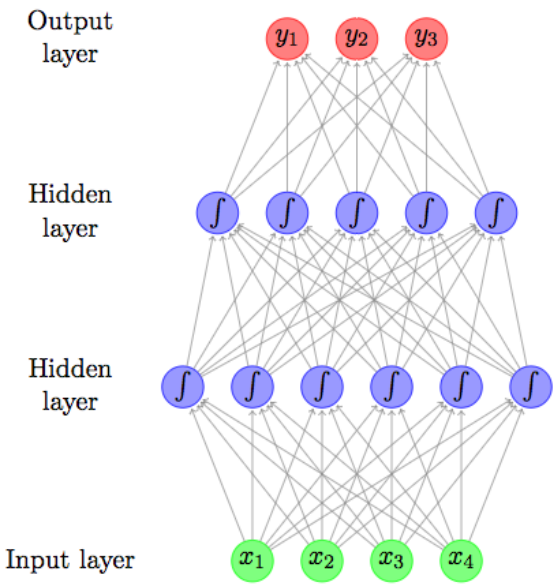
$\mathbf{x} \in \mathbb{R}^{d_{in}}, \mathbf{W}^1 \in \mathbb{R}^{d_{in} \times d_1}, \mathbf{b}^1 \in \mathbb{R}^{d_1}, \mathbf{W}^2 \in \mathbb{R}^{d_1 \times d_2}, \mathbf{b}^2 \in \mathbb{R}^{d_2},$

- Alternatively:

$$NN_{MLP2}(\mathbf{x}) = y$$
$$\mathbf{h}^1 = g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)$$
$$\mathbf{h}^2 = g^2(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2)$$
$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3$$

- each $\mathbf{W}^i \in \mathbb{R}^{d_{in} \times d_{out}}$ represents embeddings of the data in d_{out} dimensions

Multilayer Feed-forward NNs



$$NN_{MLP2}(\mathbf{x}) = y$$
$$\mathbf{h}^1 = g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)$$
$$\mathbf{h}^2 = g^2(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2))$$
$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3$$

Common non linear functions

- Sigmoid ($x \rightarrow [0, 1]$) $\sigma(x) = \frac{1}{1+e^{-x}}$
- Hyperbolic tangent (tanh) S-shaped function, $x \rightarrow [-1, 1]$

$$tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

- Hard tanh: approximation of the tanh function faster to compute.

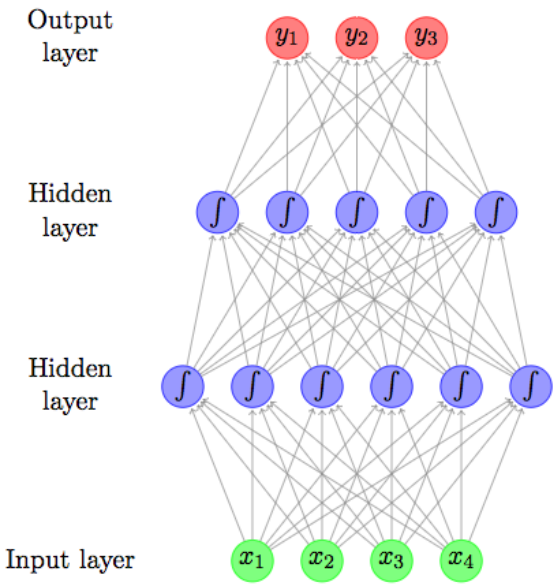
$$hardtanh(x) = \begin{cases} -1 & x < -1 \\ 1 & x > 1 \\ x & otherwise \end{cases}$$

- Relu:

$$Relu(x) = max(0, x) = \begin{cases} 0 & x < 0 \\ x & otherwise \end{cases}$$

ReLU > tanh > sigmoid

Multilayer Feed-forward NNs



$$y = NN_{MLP2}(\mathbf{x})$$
$$\mathbf{h}^1 = g^1(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1)$$
$$\mathbf{h}^2 = g^2(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2))$$
$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3$$

For classification, we use softmax:

$$\mathbf{y} = [y_1, \ldots, y_3]$$
$$softmax(y_i) = \frac{e^{y_i}}{\sum_{j=1}^{k} e^{y_j}}$$

- vector of non-negative real numbers that sum to 1
- Softmax transformation used when we need a probability distribution over the possible output classes.

Softmax

- Change the activation function in the output layer:

$$a_j^L = \frac{e^{z_j^L}}{\sum_k e^{z_k^L}}$$

  - $0 \leq a_j^L \leq 1$
  - Denominator: sum over all (k) output neurons

- Output is a probability distribution

$$\sum_j a_j^L = \frac{\sum_j e^{z_j^L}}{\sum_k e^{z_k^L}} = 1$$

  - sigmoid would not produce it

Deep Learning - Hyperparameters tuning - Preliminaries

- consider a deep neural network on a training dataset $\{(x_i, y_i)\}_{i=1,\dots,N}$

$$F(\theta, x_i) = \hat{y}_i \text{ and } L(\theta) = \frac{1}{N}\sum_{i=1}^{N} l(\hat{y}_i, y_i)$$

- Parameters $\theta = w$: parameters of the network, F: function representing the network, L : loss funtion of the network and l: loss for each training point.
- Typical hyperparameters:
  - learning rate
  - Batch size for SGD
  - number of units in the hidden layer(s)
  - type of activation functions